

```
-- JA estavamos usando isso o tempo, o proprio string eh um
Type String = [Char]
type Produto =(Integer,String,Double)
type Cliente = (Integer, String, Double)

type Assoc k v = [(k, v)]
find :: Eq k => k -> Assoc k v -> v
find k t = head [v | (k', v) <- t, k' == k] -- vou procurano
meu array associativa - toda dyupla que estiver igual a k quero
ele

preco :: Produto -> Double
preco (_, _, p) = p

pago:: Cliente -> Double
pago (_, _, p) = p

atualizaPreco :: Produto -> Double -> Produto
atualizaPreco (idProd, nome,preco) inflacao = (idProd, nome,
preco * (1 + inflacao))

troco :: Produto -> Cliente-> Double
troco p c = abs( pago c - preco p )

-- Tipo somma

-- Bool, True e False já existem no Prelude. Não precisamos
redefini-los.

data Dir = Norte | Sul | Leste | Oeste deriving Show

--deriving: da a possibilidade de imprimir esse tipo

type Coord = (Int, Int)
type Passo = Coord -> Coord

para :: Dir -> Passo
para Norte (x,y) = (x, y + 1)
para Sul (x,y) = (x, y - 1)
para Leste (x,y) = (x + 1, y)
para Oeste (x,y) = (x - 1, y)

caminhar :: [Dir] -> Passo
caminhar ds coord = foldl (flip para) coord ds
-- foldl nada mais eh do que isso aqui
-- caminhar [] coord = coord
-- caminhar (d:ds) coord = caminhar ds (para d coord)
```

```
-- Tipo Produto

data Ponto = MkPonto Double Double deriving Show
-- Cada valo do tipo ponto vi conte dois valores do tipo
double

dist :: Ponto -> Ponto -> Double
dist (MkPonto x y ) (MkPonto x' y') = sqrt $ (x-x')^2 + (y-
y')^2

data Forma = Circulo Ponto Double | Retangulo Ponto Double
Double deriving Show

quadrado :: Ponto -> Double -> Forma
quadrado p l = Retangulo p l l

-- Tipos parametrizados

data Identidade a = Id a
-- o tipo ponto precisa definir um ponto a e ele vai guardar 2
valores desse tipo dependente do que eh esse tipo

-- Maybe, Just e Nothing já existem no Prelude.

maybeDiv :: Int -> Int -> Maybe Int
maybeDiv _ 0 = Nothing
maybeDiv x y = Just (x `div` y) -- se y for 0 ele nao vai
retornar nada, se nao ele vai retornar o valor da divisao

maybeHead :: [a] -> Maybe a
maybeHead [] = Nothing
maybeHead (x:xs) = Just x

divComErro :: Int -> Int -> Int
divComErro m n =
  case maybeDiv m n of
    Nothing -> error "Divisao por zero"
    Just x -> x

-- data Either a b = Left a | Right b deriving Show (tipoa
aparametrico acita dois tirpo e aceita dois tipos e pode ser um
ou outro )

eitherDiv :: Int -> Int -> Either String Int
eitherDiv _ 0 = Left "Divisao por zero"
eitherDiv x y = Right (x `div` y)
-- Fuzzy = verdadeiro falso pertinencia double
--fuzzy fica == False se o vamor menor ou igauila ser veredri
maior ou iguakl a zero
```

```
data Fuzzy = Verdadeiro | Falso | Pertinencia Double deriving
Show

fuzzyfica :: Double -> Fuzzy
fuzzyfica x
  | x <= 0 = Falso
  | x >= 1 = Verdadeiro
  | otherwise = Pertinencia x

f :: Identidade a -> a
f (Id x) = x
-- Tipo Recursivo
--Tipos que sao fefivndos udando o mprorpro tipo que estamos
definindo

data Lista a = Vazia | Cons a (Lista a) deriving Show
-- cons 1 (Cons 2 (Cons 3 Vazia)) == [1,2,3]

-- Uma arvore binaria pode ser uma folha com um valor (Leaf)
-- ou um no (Node) com um valor e duas subarvores: esquerda e
direita.
-- O tipo e recursivo porque cada subarvore tambem e uma Tree
a.
data Tree a = Leaf a | Node {_left :: Tree a,
                             _value :: a,
                             _right :: Tree a
                           } deriving Show

t :: Tree Int
t = Node t1 5 t2
  where
    t1 = Node (Leaf 1) 3 (Leaf 4)
    t2 = Node (Leaf 6) 7 (Leaf 8)
{-
      5
     /\
    3  7
   /\ /\
  1 4 6 8
-}

contem :: Ord a => Tree a -> a -> Bool
contem (Leaf y) x = x == y
contem (Node l y r) x | x == y = True
                      | x < y = contem l x
                      | x > y = contem r x

-- Record Type
data Ponto3D = Ponto { coordX :: Double, coordY :: Double,
coordZ :: Double } deriving Show
```

```
-- No lugar de um loop igual na linguagem
semantica, usamos recursão para percorrer
listas e realizar operações. Abaixo estão
algumas funções que demonstram o uso de
recursão em Haskell.
--Nao pode ter chamada infinita. por isso é
importante ter uma condição de parada.

-- Funcao recursirva eh uma funcao definida em
termoa dela memsi possui tres partes
-- → PARTICAO DO PROBLEMA EM SUBPRONMAS
-- → a base final da recursao quando os
problemas sao taoa pquenos que podem ser
resolvidos diretamente, geralne de maneira
trivial (caso base)
-- → A combinacao da repostara parciai,
formando a reposta toptal

fatorial :: Integer → Integer
fatorial n
  | n == 0 = 1 -- caso base
  | n > 0 = n * fatorial (n-1) -- caso
recursivo, a funcao se chama a si mesma com um
valor menor de n

-- Recursao em calda de funções auxiliares,
como 'elem', 'nub', 'isAsc', e 'hasPath',
permite que você manipule listas e verifique
condições de maneira eficiente. Cada função é
definida com base em casos base e casos
recursivos, garantindo que a execução termine
corretamente.
-- escreva um funsao recuersdiva calcule o
resto inteiro da divisao de dois numeros
utilizanso subtacoes sucessivas

divRec :: Int → Int → Int
divRec a b
  | b > a = a -- caso base trivial, quando o
divisor é maior que o dividendo, o resto é o
próprio dividendo
  | b == a = 0 --caso base trivial
  | otherwise = divRec (a - b) b -- caso
recursv diminuinodo o tamanho do preblema

-- 1) Escreva uma função recursiva que receba
como parâmetros dois números inteiros
positivos, x e n, e retorne o resultado de
x^n, realizando somas sucessivas.
-- Exemplo: mult 3 4 = 3 + 3 + 3 + 3 = 12
mult :: Int → Int → Int
mult x n
  | n == 0 = 0
  | otherwise = x + mult x (n - 1) -- caso
recursivo: soma x repetidas vezes

-- 2) Escreva uma função recursiva para o
cálculo do MDC (máximo divisor comum) de dois
números inteiros.
-- Veja a definição recursiva:
-- mdc(a, b) = b, se a % b == 0
-- mdc(a, b) = mdc(b, a % b), caso contrário
```

```
haskell otimiza chamas com recucao em cauda de
maaina denonomizar recursos e aumentar
eficieima

-- Listas
-- Tipo das listas entre colchetes
-- Lista de listas = [[1,2], [3,4]] [[Int]]

-- Preenchemeneto → 1..10 1,3..10 10,8
.. 8

-- : que toma um elemento e uma lista que
pode estar vazia como seus argumentos

-- head e tail → head retorna o primeiro
elemento da lista, tail retorna a lista sem o
primeiro elemento
-- ++ concatenacao de listas, concatena duas
listas em uma só

compr :: [Int] → Int -- função que calcula o
comprimento de uma lista de inteiros
compr [] = 0 -- caso base: lista vazia tem
comprimento 0
compr (h:t) = 1 + compr t -- caso recursivo:
soma 1 ao comprimento da cauda da lista

-- Tornando-se por base o operador de
construcao (:), lista [1,2,3] pode ser escrita
como 1:[2,3], onde 1::Int eh a cabeca da lista
(head) e a lista [2,3]::[Int] eh a sua cauda
(tail). A funcao compr percorre a lista
recursivamente, contando os elementos ate
chegar na lista vazia, que eh o caso base da
recursao.

-- uma eh vazia ou eh yma lista com cabeca
seguida de cauda o exempleo a seguir implemena
uma funcao que calcula o cumprimento de uma
lista de inteiros

-- fazer com hull e gurda

cubo :: Int → Int
cubo n = n * n * n -- função que calcula o
cubo de um número inteiro

aoCubo :: [Int] → [Int] -- funcão que recebe
uma lista de inteiros e retorna uma lista com
os cubos desses inteiros
aoCubo [] = [] -- caso base: lista vazia
retorna lista vazia
aoCubo (h:t) = cubo h : aoCubo t -- caso
recursivo: aplica a função cubo à cabeca da
lista e concatena com o resultado da chamada
recursiva à cauda da lista

-- Escreva uma funcao em Haskell que calcule a
somatoria so elementos de um vetor de
inteiros.
somatoria :: [Int] → Int
somatoria [] = 0
somatoria (h:t) = h + somatoria t

-- Esrecva uma funcao e, haskell que verifique
se uma strino possuir o caractere informado
passado como parametro

temChar :: [Char] → Char → Bool
temChar [] ch = False
temChar (h:t) ch
  | h == ch = True
  | otherwise = temChar t ch
```

```
-- Escreva uma funcao em Haskell que verifique
se uma lista de inteiros possui um elemento
maior que o valor informado como parametro
ehMaior :: [Int] → Int
ehMaior f = -1 -- caso base: lista vazia é
considerada maior que qualquer
ehMaior (h:t) = if h ≥ maiorcauda then h else
maiorcauda
  where maiorcauda = ehMaior t

raizes :: Float → Float → Float → [Float]
raizes a b c | delta < 0 = []
  | delta == 0 = [(-b)/(2*a)]
  | delta > 0 = [(-b - sqrt delta)/(2*a),
(-b + sqrt delta)/(2*a)]
  where delta = b*b - 4*a*c

--mais pares = [2 * x | x ← [0..10]]

-- escreva uam funcaoem em haskell que retorner
os 10 primeiros mustiplos de n utilie o gerar
lista

tabuada :: Int → [Int]
tabuada n = [n * x | x ← [0..10] ]

-- escreva umafunacao que verifique se um
determinado numero eh primo
isprimo :: Int → Bool
isprimo n = if length [x | x ← [1..n], mod n
x ==0] == 2 then True else False

-- QuickSort eh um algoritmo de ordenacao que
utiliza a tecnica de divisao e conquista, onde
o problema eh dividido em subproblemas
menores, resolvidos recursivamente e
combinados para obter a solucao final. O
QuickSort escolhe um elemento como pivô e
particiona a lista em duas sublistas: uma com
elementos menores que o pivô e outra com
elementos maiores. Em seguida, aplica-se
recursivamente o QuickSort nas sublistas e
combina os resultados.
-- 1. escolher um elemento do vetor conhecido
como pivô
-- 2. patriciona o verto de maneira que todos
os elementos anteriores ao pivô sejam menor
que ele, e todos os elementos posteriores
sejam maiores
-- 3. Ordena recursivamente os vetore de
elementos menores e maios
-- a baase recursao dao os vetores de tamanho
0 ou 1 que ja se encontram ordenados
-- a casa passo de algoritmo pelo menos um
elemntoo eh colocado sua posicao definitiva, e
nao sera mais manipulado no passo seguinte
```

```
qsort :: [Int] → [Int]
qsort [] = []
qsort (h:t) = qsort [y | y < t, y < h ]
  ++[h]
  ++ qsort [y |
y < t, y ≥ h]

-- Uma tupla, pode der mais de um tipo, eh
cocecaod e valores
--- os valores sao colocados entre parentese e
separados p9or uam virgula ("John", 123456)

--valores de tuplkas sao definidos de maneiras
semelhantes as listas mas com a utilizao de
parentese ao inves de colces alem disso uma
tupla pode conter valores de tipos de
diferentes enouando uma lista nao a ordem
dosnelementos importa de maneira que a tupla
```

```
(12345,"Jhon") eh diferente da dupla
modstreadsa acima
-- tupla de par funcoes fst para o priemeiro e
o snd para o segundo
type NomeAluno = String
type MediaNota = Int
type Aluno = (NomeAluno, MediaNota)
type Turma = [Aluno]

-- NOvo nome a tipos que existea para ficar
mais legivel ainda

aprovado :: Turma → Int → [NomeAluno]
aprovado tma nota = [nome | (nome, media) ←
tma, media ≥ nota]

-- crie a respesenracaode um pontod e teres
simensoes a representacao deve ser realizada
por meio de uma tupla e a definicao de um novo
tipo
-- escreva uma funcao que caclule a distancia
entre dois pontos como arfumentos

type Ponto = (Float, Float, Float)

distancia (x1,y1,z1) (x2,y2,z2) = sqrt (dx*dx
+ dy*dy + dz*dz)
  where
    dx = x1-x2
    dy = y1 -y2
    dz =z1-z2

-- Matchinh patterns nocaio de casemnto de
padroes ao chamaa funcaoed pasanod um
parametro ele tentara encaixar em qual padrao
ele encaiza de cima para baixo os padroes
fornecidos pelo prooramador. o primeiro a ser
encontrado eh execeutado com o valor passado
como parametro

-- de cima pra baixo o continga deve ser por
ultimo
padroes :: Int → String
padroes 1 = "Um"
padroes _ = "Nao esta entre 1 e 3 "

padroes2 :: [Int] → Int
padroes2 [] = 0
padroes2 (_,t) = padroes2 t

type Tupla4 = (Int,Int,Int,Int)
padroes3 :: Tupla4 → String
padroes3 (_,_,_,fourth) | fourth > 10 = "Maior
que 10."
  | otherwise = "Nao eh

maior que 10"

opp2 :: (Int, (Int,Int)) → Int
opp2 (1, (a,b)) = a + b
opp2 (2, (a,b)) = a - b
opp2 _ = 0

-- Funcoes de alta ordem

-- fucoes de alta ordem sao aquela que operam
sobre outras funcoes ou seja que podem receber
outra funcao como argumenro ou retornar uma
funcao como resultstultaso

dobra :: Int → Int
dobra n = n + n

quadrado :: Int → Int
quadrado n = n*n
-- uma funcao
```

```
mapInt :: (Int → Int) → [Int] → [Int]
mapInt _ [] = []
mapInt f (h:t) = (f h) : (mapInt f t)

-- eu aponto para funcao e no termina eu chamo
a funcao passando ela por parametro

-- 0 que eh filtri , que recebe como agrumenro
ua funcao de teste e seleciona os elementos da
lista que satisfazem a condicao desejada

filtro :: (Int → Bool) → [Int] → [Int]
filtro f [] = []
filtro f (h:t)
  | (f h) == True = h : (filtro f t)
  | otherwise = filtro f t

pares :: Int → Bool
pares x = (mod x 2 == 0)

impares :: Int → Bool
impares x = (mod x 2 == 1 )

-- uma funcao que busca o maior elemento em
uma lista de inteiros com maior e menor
comofara isso

busca :: (Int → Int → Int ) → [Int] → Int
busca _ [] = 0 -- caso base, se a lista
estiver vazia, retorna 0
busca _ [h] = h -- caso base, se a lista tiver
apenas um elemento, retorna esse elemento
busca f (h:t) = f h (busca f t) -- se eu tiver
cabeca e cauda na lista eu aplico a funcao f
na cabeca e no resultado da chamada recursiva
na cauda da lista

maior :: Int → Int → Int
maior a b = if a > b then a else b

menor :: Int → Int → Int
menor a b = if a < b then a else b

--
maiorElemento :: Int → Int → Bool
maiorElemento a b = if a > b then True else
False

menorElemento :: Int → Int → Bool
menorElemento a b = if a < b then True else
False

buscaLista :: (Int → Int → Bool) → [Int] →
Int
buscaLista f [] = -1
buscaLista f [h] = h
buscaLista f (h:t) = if f h (buscaLista f t)
then h else (buscaLista f t)

-- A mesma busca, guardando em x o resultado
encontrado na cauda.
buscaListaWhere :: (Int → Int → Bool) →
[Int] → Int
buscaListaWhere _ [] = -1
buscaListaWhere _ [cab] = cab
buscaListaWhere f (cab:cauda) =
  if f cab x then cab else x
  where
    x = buscaListaWhere f cauda
```